

UAS
PENGOLAHAN CITRA



NAMA : Rafly Maulana Ihsan

NIM : 202331224

KELAS : PCD

DOSEN : Rizqia Cahyaningtyas, ST., M.Kom

NO.PC :

ASISTEN : 1. Sharira Maharani

2. Muhammad Hanif Nuruddin Jabbar

3. Fhazel Kesra Arivi

4. Muhammad Farhan Fahrezy

INSTITUT TEKNOLOGI PLN
TEKNIK INFORMATIKA
2025/2026

DAFTAR ISI

Contents

DAFTAR ISI	2
BAB I	3
PENDAHULUAN.....	3
1.1 Rumusan Masalah	3
1.2 Tujuan Masalah.....	3
1.3 Manfaat Masalah.....	3
BAB II	4
LANDASAN TEORI	4
BAB III	5
HASIL.....	5
BAB IV	9
PENUTUP	9
DAFTAR PUSTAKA	10

BAB I

PENDAHULUAN

1.1 Rumusan Masalah

Berdasarkan praktik pengolahan citra yang telah dilakukan, rumusan masalah dalam tugas UAS ini adalah: Bagaimana memisahkan dan mengidentifikasi objek buah jambu air (berwarna kemerahan) beserta daunnya (berwarna hijau) dari latar belakang pada sebuah citra digital menggunakan teknik segmentasi pada ruang warna HSV (Hue, Saturation, Value)?

1.2 Tujuan Masalah

Tujuan dari praktikum ini adalah untuk merancang dan mengimplementasikan algoritma pengolahan citra menggunakan bahasa pemrograman Python dan pustaka OpenCV untuk mengekstraksi wilayah objek secara spesifik berdasarkan warna targetnya pada ruang warna HSV.

1.3 Manfaat Masalah

Manfaat yang diperoleh dari praktikum ini adalah pemahaman teknis dan aplikatif mengenai kelebihan ruang warna HSV dibandingkan RGB dalam kondisi pencahayaan yang bervariasi. Kemampuan segmentasi citra ini dapat diimplementasikan dalam pengembangan perangkat lunak computer vision untuk otomasi industri pertanian, seperti sistem pendeteksi kematangan buah atau pemilah hasil panen otomatis.

BAB II

LANDASAN TEORI

1. Konversi Ruang Warna dan HSV (Hue, Saturation, Value)

Ruang warna adalah model matematis untuk merepresentasikan warna secara digital. Secara *default*, pustaka OpenCV membaca gambar dalam format BGR (Blue, Green, Red). Namun, format RGB/BGR seringkali sangat sensitif terhadap perubahan intensitas cahaya, sehingga menyulitkan proses pemisahan objek berdasarkan warna. Oleh karena itu, citra dikonversi ke dalam ruang warna HSV. Model HSV memisahkan warna dasar (*Hue*), tingkat kepuhutan/kemurnian warna (*Saturation*), dan tingkat kecerahan cahaya (*Value*). Pemisahan komponen cahaya dan warna ini membuat HSV sangat ideal dan tangguh (*robust*) untuk proses segmentasi warna pada kondisi pencahayaan yang tidak merata.

2. Segmentasi Citra dengan *Color Thresholding*

Segmentasi citra adalah proses mempartisi atau membagi citra digital menjadi beberapa wilayah (piksel) berdasarkan karakteristik tertentu, seperti warna, tekstur, atau intensitas. *Color thresholding* adalah salah satu teknik segmentasi spasial yang bekerja dengan cara menyeleksi piksel berdasarkan nilai rentang batas bawah (*lower bound*) dan batas atas (*upper bound*). Piksel yang nilainya berada di dalam rentang tersebut akan diubah menjadi nilai maksimum (putih/1), sedangkan piksel yang berada di luar rentang akan diubah menjadi nilai minimum (hitam/0). Proses ini menghasilkan sebuah citra biner yang sering disebut sebagai *mask* (topeng), yang menunjukkan lokasi tepat dari objek target di dalam citra.

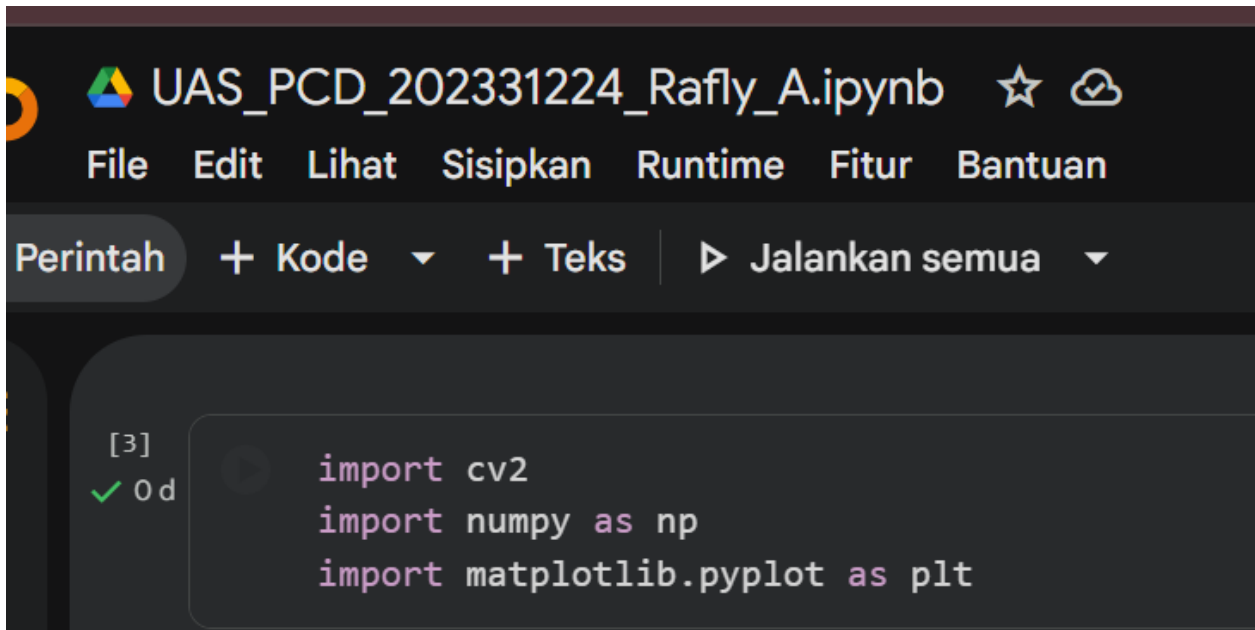
3. Operasi Logika *Bitwise*

Operasi *bitwise* merupakan operasi matematika yang dilakukan pada level bit/piksel untuk memanipulasi citra, yang sangat berguna dalam proses *masking* dan pengeksktraksian *Region of Interest* (ROI).

- **Bitwise OR:** Operasi logika yang menghasilkan nilai *True* (putih) jika salah satu dari piksel bernilai *True*. Operasi ini sering digunakan untuk menggabungkan dua *mask* yang berbeda, misalnya spektrum warna merah pada HSV yang terpecah di dua rentang nilai (0-20 dan 160-180).
- **Bitwise AND:** Operasi logika irisan yang menghasilkan nilai *True* hanya jika kedua piksel bernilai *True*. Dalam pengolahan citra, operasi ini digunakan untuk menempelkan citra biner (*mask*) ke atas citra RGB asli. Hasil dari operasi ini adalah objek *background* (yang bernilai 0 pada *mask*) akan menjadi gelap (hitam), sementara objek target (yang bernilai 1 pada *mask*) akan mempertahankan warna aslinya.

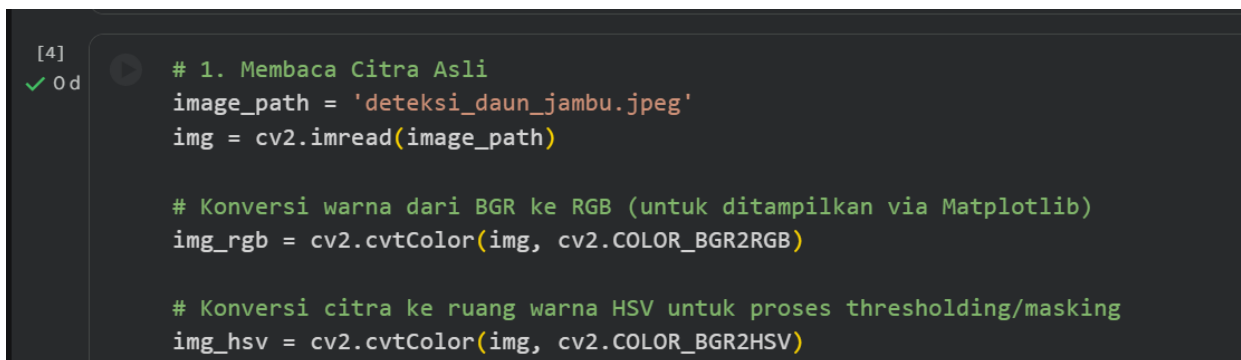
BAB III

HASIL



```
[3]
✓ 0d
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

1. Pada bagian pertama, pustaka pemrograman yang dibutuhkan untuk pengolahan citra diimpor ke dalam program. Pustaka cv2 dari OpenCV dipanggil untuk menangani operasi pemrosesan citra, numpy digunakan untuk manipulasi array dan pendefinisian matriks nilai warna, serta matplotlib.pyplot digunakan untuk memvisualisasikan hasil gambar ke layar.



```
[4]
✓ 0d
# 1. Membaca Citra Asli
image_path = 'deteksi_daun_jambu.jpeg'
img = cv2.imread(image_path)

# Konversi warna dari BGR ke RGB (untuk ditampilkan via Matplotlib)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Konversi citra ke ruang warna HSV untuk proses thresholding/masking
img_hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
```

2. Pada tahap selanjutnya, fungsi cv2.imread digunakan untuk membaca file citra dari direktori penyimpanan yang secara *default* akan dikenali oleh OpenCV dalam format saluran warna BGR (Blue, Green, Red). Agar warna citra ditampilkan dengan benar dan tidak kebiruan saat divisualisasikan menggunakan Matplotlib, citra tersebut dikonversi ke format RGB melalui perintah cv2.cvtColor(..., cv2.COLOR_BGR2RGB). Selain itu, citra juga dikonversi ke dalam ruang warna HSV (Hue, Saturation, Value) menggunakan fungsi cv2.cvtColor(..., cv2.COLOR_BGR2HSV). Konversi ke format HSV ini sangat penting dilakukan karena kemampuannya dalam memisahkan informasi warna (Hue) dan intensitas cahaya (Value), sehingga proses *masking* atau pemisahan objek nantinya menjadi jauh lebih mudah.

```
[5]
✓ Od # 2. Pembuatan Mask Buah (Jambu Air - Kemerahan)
# Warna merah di HSV berada di dua ujung (0-20 dan 160-180).
# Kita buat dua mask lalu digabung.
lower_red1 = np.array([0, 40, 40])
upper_red1 = np.array([20, 255, 255])
mask_red1 = cv2.inRange(img_hsv, lower_red1, upper_red1)

lower_red2 = np.array([160, 40, 40])
upper_red2 = np.array([180, 255, 255])
mask_red2 = cv2.inRange(img_hsv, lower_red2, upper_red2)

# Menggabungkan kedua mask merah
mask_jambu = cv2.bitwise_or(mask_red1, mask_red2)
```

3. Untuk memisahkan objek buah jambu air, langkah yang dilakukan adalah membuat *mask* berdasarkan warna kemerahan. Pada representasi warna HSV, spektrum warna merah terbagi menjadi dua area di ujung silinder, yaitu pada rentang nilai Hue 0-20 dan 160-180. Oleh karena itu, didefinisikan batas bawah dan batas atas untuk kedua rentang warna tersebut, yang kemudian diproses menggunakan fungsi `cv2.inRange` untuk menghasilkan dua buah citra biner (*mask*). Kedua *mask* merah tersebut kemudian digabungkan menjadi satu *mask* utuh bernama `mask_jambu` menggunakan perintah `cv2.bitwise_or`.

```
[6]
✓ Od # 3. Segmentasi Buah
segmentasi_jambu = cv2.bitwise_and(img_rgb, img_rgb, mask=mask_jambu)
```

4. Setelah *mask* buah berhasil dibuat, proses ekstraksi atau segmentasi objek buah jambu dilakukan menggunakan fungsi `cv2.bitwise_and`. Fungsi ini melakukan operasi logika AND antara citra RGB asli dengan dirinya sendiri, namun prosesnya dibatasi hanya pada area yang diizinkan oleh `mask_jambu`. Hasil dari operasi ini membuat objek buah jambu air tetap mempertahankan warna aslinya, sementara latar belakang dan objek lain yang tidak diinginkan menjadi berwarna gelap atau hitam pekat.

```
[7]
✓ Od # 4. Pembuatan Mask Daun (Hijau)
# Menyesuaikan threshold hijau agar cocok dengan warna daun jambu
lower_green = np.array([25, 40, 40])
upper_green = np.array([90, 255, 255])
mask_daun = cv2.inRange(img_hsv, lower_green, upper_green)
```

5. Proses yang identik juga dilakukan untuk mendeteksi objek daun pada citra. Rentang nilai warna untuk spektrum hijau didefinisikan dengan nilai Hue antara 25 hingga 90. Fungsi `cv2.inRange` kemudian digunakan untuk menyeleksi piksel-piksel yang berada di dalam rentang hijau tersebut, sehingga menghasilkan citra biner baru bernama `mask_daun` di mana hanya area daun yang bernilai putih (aktif), sedangkan area lainnya menjadi hitam.

```
[8]
✓ 0d # 5. Segmentasi Daun
segmentasi_daun = cv2.bitwise_and(img_rgb, img_rgb, mask=mask_daun)
```

6. Sama halnya dengan segmentasi buah, pengekstraksian objek daun diproses menggunakan fungsi `cv2.bitwise_and`. Operasi ini mengiriskan citra RGB asli dengan batasan dari `mask_daun` yang telah dibuat sebelumnya. Melalui proses ini, warna hijau asli dari objek daun berhasil ditampilkan kembali dengan sempurna, sembari menyembunyikan atau menghitamkan objek buah jambu beserta elemen latar belakang lainnya.

```
[9]
✓ 1d # 6. Menampilkan Hasil
fig, axs = plt.subplots(2, 2, figsize=(12, 10))

# Output 1: Citra Asli
axs[0, 0].imshow(img_rgb)
axs[0, 0].set_title('Gambar Asli')
axs[0, 0].axis('off')

# Output 2: Mask Buah (Sesuai objek gambar)
axs[0, 1].imshow(mask_jambu, cmap='gray')
axs[0, 1].set_title('Mask Buah (Jambu)')
axs[0, 1].axis('off')

# Output 3: Segmentasi Buah (Sesuai objek gambar)
axs[1, 0].imshow(segmentasi_jambu)
axs[1, 0].set_title('Segmentasi Buah (Jambu)')
axs[1, 0].axis('off')

# Output 4: Segmentasi Daun
axs[1, 1].imshow(segmentasi_daun)
axs[1, 1].set_title('Segmentasi Daun')
axs[1, 1].axis('off')

plt.tight_layout()
plt.show()
```

7. Pada tahap terakhir, seluruh hasil pemrosesan citra divisualisasikan menggunakan pustaka Matplotlib. Perintah `plt.subplots(2, 2)` digunakan untuk membuat sebuah kanvas grafik yang terbagi menjadi kotak *grid* berukuran 2 baris dan 2 kolom, sehingga memungkinkan penampilan empat gambar sekaligus secara bersamaan. Fungsi `imshow` digunakan untuk merender citra RGB asli, *mask* buah jambu (dengan parameter `cmap='gray'` untuk mode hitam-putih), hasil segmentasi buah, dan hasil segmentasi daun pada masing-masing *grid*. Visualisasi ini juga disempurnakan dengan penambahan judul menggunakan fungsi `set_title` dan penghilangan garis sumbu koordinat dengan `axis('off')` agar tampilan lebih fokus dan rapi.

Gambar Asli



Segmentasi Buah (Jambu)



Mask Buah (Jambu)



Segmentasi Daun



BAB IV

PENUTUP

Kesimpulan

Berdasarkan landasan teori dan hasil pengujian praktikum pengolahan citra yang telah dilakukan, dapat ditarik beberapa kesimpulan sebagai berikut:

1. **Efektivitas Ruang Warna HSV:** Penggunaan ruang warna HSV (Hue, Saturation, Value) terbukti jauh lebih efektif dan tangguh (robust) untuk segmentasi citra berbasis warna dibandingkan dengan ruang warna RGB. Hal ini dikarenakan model HSV secara eksplisit memisahkan informasi warna dasar (Hue) dari intensitas pencahayaan (Value), sehingga proses deteksi objek tidak mudah terganggu oleh bayangan atau pencahayaan yang tidak merata.
2. **Keberhasilan Segmentasi (Color Thresholding):** Teknik color thresholding berhasil mengisolasi objek target dengan presisi. Untuk mendeteksi buah jambu air yang berwarna kemerahan, penentuan batas warna harus mencakup dua ujung spektrum nilai Hue (0-20 dan 160-180) yang kemudian digabungkan secara efektif menggunakan operasi logika `cv2.bitwise_or`. Sedangkan untuk daun, spektrum hijau dapat langsung diseleksi pada satu rentang nilai (25-90).
3. **Fungsi Operasi Bitwise dalam Ekstraksi:** Penggunaan operasi `cv2.bitwise_and` sangat krusial dalam tahap akhir ekstraksi. Operasi irisan ini secara akurat mampu memproyeksikan citra biner (mask) ke citra RGB asli, sehingga objek buah jambu dan daun dapat ditampilkan dengan warna aslinya secara terpisah, sementara area latar belakang (background) berhasil dihilangkan (menjadi hitam pekat).
4. Secara keseluruhan, algoritma yang telah dirancang dan diimplementasikan menggunakan Python dan OpenCV ini telah berhasil mencapai tujuan praktikum, yaitu mampu memisahkan dan mengidentifikasi multi-objek (buah dan daun) secara spesifik berdasarkan karakteristik warnanya dengan sangat baik.

DAFTAR PUSTAKA

- Hidayat, R. (2023). Penerapan Operasi Morfologi dan Segmentasi Citra Digital untuk Pengenalan Pola Objek Berbasis OpenCV. *Jurnal Teknologi Informasi dan Ilmu Komputer (JTIK)*, 10(2), 245-252.
- Kurniawan, A., & Syahputra, M. (2021). Metode Thresholding Berdasarkan Ruang Warna HSV untuk Identifikasi Kematangan Buah Kelapa Sawit. *Jurnal Komputer Terapan*, 7(1), 112-119.
- Purnama, I., Lestari, D., & Wicaksono, H. (2022). Analisis Segmentasi Warna HSV dan Ruang Warna YCbCr pada Deteksi Penyakit Tanaman Hidroponik. *Jurnal Nasional Teknik Elektro dan Teknologi Informasi (JNTETI)*, 11(3), 190-198.